TECHNICAL UNIVERSITY OF DENMARK

Exercise 5
Expectation-maximization algorithm

Date of submission: 20th of October, 2025

Submitted By:

Holger Max Fløe Lyng, S214776

Maja Rebien Johannesen, S204287

Dorthea Nørregaard, S204295

Othilia Wagner, S214780

Philip Vestergaard-Laustsen, s196248

Table of Contents

Introduction

Segmentation of medical images is important for both the individual patient, e.g., diagnosis and treatment plan, and for group studies, e.g., drug trials.
To do this the generative model can be used. It tries to describe how an image could be generated. In other words we simulate the imaging process. The generative model can be divided into two parts:

- Labeling model
- Imaging model

The **labeling model** describes where different anatomical structures would appear in the image. In this model, each voxel is assigned a label, and these labels are represented by a probability distribution $p(l|\theta_l)$, where l describes a vectorized label image, and θ_l are specific parameters that are known or need to be estimated. The probability function describes how likely specific structures would appear in specific places.
The **imaging model** describes what image intensities we would expect to see, given the labels. It is written as $p(d|l, \theta_d)$, where d is the observed intensity image and θ_d are the parameters given or estimated.

By using Bayes' rule, we can compute the posterier probability of how likely is each possible label configuration:

$$p(l \mid d, \hat{\theta}) = \frac{p\left(d \mid l, \hat{\theta}_d\right) p\left(l \mid \hat{\theta}_l\right)}{p(d \mid \hat{\theta})} \tag{1}$$

One example of a simple generative model is the **Gaussian-Mixture Model** (GMM). This model assumes that our data comes from a mixture of different Gaussian distributions. So this means it assumes that each tissue or structure can be represented with an individual gaussian distribution. So each label k has:

- Mean intensity μ_k
- Variance σ_k^2

- Probability of appearing (mixing weight) π_k

In the GMM the labeling model describes that each voxel belongs to the tissue type k with probability π_k. Here the voxels do not depend on each other, meaning there is no spatial relationship.

$$p\left(l \mid \theta_l\right) = \prod_{n=1}^{N} p\left(l_n \mid \theta_l\right) \tag{2}$$

The imaging model describes that given a voxel n label, its intensity will follow a Gaussian distribution, meaning its intensity will be near the mean μ_k with a variability σ_k^2.

$$p\left(d_n \mid l_n = k, \theta_d\right) = \mathcal{N}\left(d_n \mid \mu_k, \sigma_k^2\right) \tag{3}$$

In the GMM parameters $\theta = \{\mu_k, \sigma_k^2, \pi_k\}$ can be manually picked, but this is not practical in MRI as if you keep the segmentation parameters fixed for the T1 image they wouldn't work on a T2 image, as the intensity values vary. To estimate the parameters automatically we look for the parameters that maximize the likelihood function $p(d|\theta)$, that describes the probability of the image d for the different settings of the parameter θ. For simplifying the math and easier computation we use the log-likelihood:

$$\hat{\theta} = \arg\max_{\theta} \log p(d \mid \theta) \tag{4}$$

To make the optimization easier the Expectation-Maximization (EM) algorithm is used. This builds and maximizes a simpler lower bound function $Q(\theta|\tilde{\theta})$ of the log-likelihood, step by step. The lower bound is simpler and easier to maximize, and it will touch the real log-likelihood at the current parameter estimate, and stay below everywhere else. So when we maximize $Q(\theta|\tilde{\theta})$ we are guaranteed to move in the direction that increases the real log-likelihood. The lower bound can be described as follows:

$$\begin{aligned}\sum_n \left[\sum_k w_k^n \log\left(\frac{\mathcal{N}\left(d_n \mid \mu_k, \sigma_k^2\right)\pi_k}{w_k^n}\right)\right] = &-\frac{1}{2}\sum_k\left[\frac{1}{\sigma_k^2}\sum_n w_k^n\left(d_n-\mu_k\right)^2 + \left(\sum_n w_k^n\right)\log\sigma_k^2\right] \\ &+\sum_k\left[\left(\sum_n w_k^n\right)\log\pi_k\right] + \mathrm{C}\end{aligned} \tag{5}$$

The EM-algorithm is divided into two steps: E-Step and M-step. The E-step is the expectation step, where we look how likely given the current parameters that voxel n belongs to class k.

$$\omega_{n,k} = \frac{\pi_k \mathcal{N}\left(d_n \mid \mu_k, \sigma_k^2\right)}{\sum_{k'} \pi_{k'} \mathcal{N}\left(d_n \mid \mu_{k'}, \sigma_{k'}^2\right)} \tag{6}$$

In the M-step we update the model parameters using the weights obtained in the E-step.

$$\begin{aligned} \mu_k &= \frac{\sum_n \omega_{n,k} d_n}{\sum_n \omega_{n,k}} \\ \sigma_k^2 &= \frac{\sum_n \omega_{n,k} \left(d_n - \mu_k\right)^2}{\sum_n \omega_{n,k}} \\ \pi_k &= \frac{\sum_n \omega_{n,k}}{N} \end{aligned} \tag{7}$$

In task 1-4 we are initializing the GMM, and from task 5-8 we optimize the parameters using the lower bound.

Disclaimer: Parts of the coding, troubleshooting, and text refinement were assisted by using AI tools (ChatGPT). The use of AI tools in this work was conducted in accordance with DTU's Honor Code and the regulations on academic integrity. All analysis, interpretations, and conclusions are the authors' own.

1 Task 1: Image data

Display the image provided in "correctedData.mat" and plot its histogram.
Hints:

- .mat data can be loaded in python as follows.
  mat = loadmat("correctedData.mat")
  data = mat["correctedData"]
- Use 100 bins for your histogram
- The mask sets all background pixels to zero. Disregard the background pixels by clipping 0.

Looking at the histogram in figure 1 we can see that we have many background pixels with intensity 0. When looking at the zoomed histogram, we can observe how we have three "bumps". This corresponds to the three main tissue types in the image: Gray matter, White matter, and CSF.

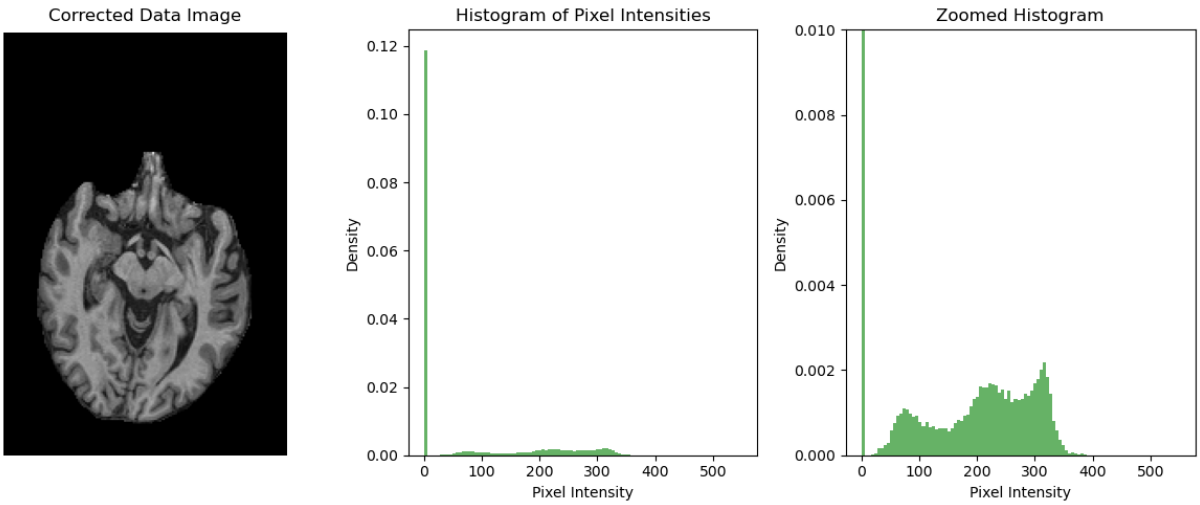

Figure 1: *Visualization of the image provided in "correctedData.mat" and of the corresponding histogram.*

2 Task 2: Initialize Gaussian-Mixture Model

Compute the minimum and maximum intensity in the image (non-zero pixels), and divide the intensity range up into three equally wide intervals. Initialize the parameters of a 3-component Gaussian mixture model by setting the means of the Gaussians to the centers of the intensity intervals, the variances to the square of the width of the intervals, and the prior weights to $\frac{1}{3}$ each.

Hint:

- In your intensity range, disregard the background pixels by clipping 0

The dataset only containing non-zero values were created in Task 1. The robust maximum is used with the 99 percentile to remove the extreme outliers. The intensities are split in three equal sizes and the mean of each interval is found. The variance is the width of the interval and will therefore be the same.

Robust max	Minimum	Means	Variances	Weigths
		71.83		
345.59	17.07	181.34	109.50	0.33
		290.84		

Table 1: *Overview of the initial values for the Gaussian Mixture model, the variances and weights are the the same for all three intervals.*

2.1 Implementation of the code

```
data_f = data.copy()
data_f= data_f[data_f > 0]
max_rm = np.nanpercentile(data_f, 99)
min = np.nanmin(data_f)

interval_width = (max_rm - min) / 3

mu = np.array([min + interval_width/2, min + 3*interval_width/2, min +
↪  5*interval_width/2])
sigma = np.array([interval_width, interval_width, interval_width])
pi = np.array([0.33, 0.33, 0.33])
```

3 Task 3: Display inital Gaussian distributions

Overlay the resulting Gaussian mixture model on the histogram by plotting each Gaussian weighted by its π_k, as well as the total weighted sum of all three Gaussian distributions (as in fig. 3.1(b) in the course notes).

Hint:

- A gaussian distribution with the parameters μ, σ^2, scaled by the weight w and evaluated at positions x can be obtained by the function
  w * scipy.stats.norm.pdf(x, mu, sigma)
- you can use the bin edges of your histogram as x.

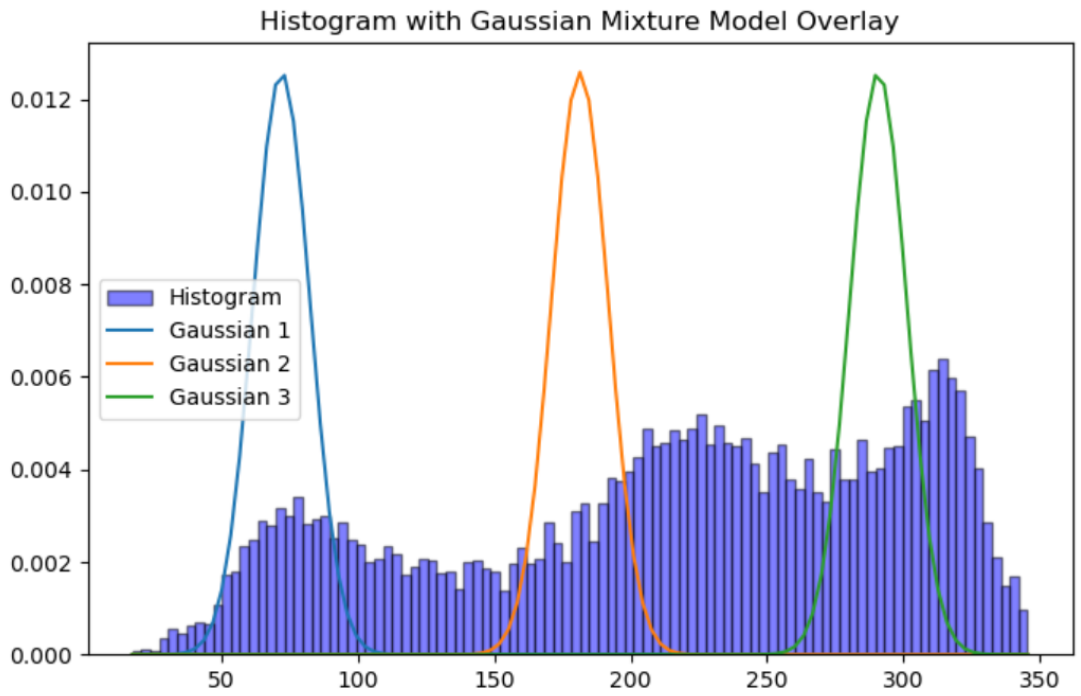

Figure 2: *Histogram of the non-zero brain intensities with the three initialized Gaussian components overlaid. Each curve corresponds to one of the Gaussian distributions defined in Task 2, scaled by its mixing weight* π_k

The Gaussian distribution is implemented using the function **`scipy.stats.norm.pdf(data, μ, σ)`**,where we use the mean, and standard deviation determined in Task 2. Figure 2 shows the Gaussian distribution plotted together with the intensity histograms. We observe, that the three Gaussian components are centered in different parts of the intensity range. They roughly follow the three visible modes of the histogram. These modes correspond to low-, medium-, and high-intensity tissue classes in the brain. So it indicates, that the initial parameter estimates from the previous task (Task 2), gives a reasonable starting point for the subsequent EM optimization.

3.1 Implementation of the code

```
1
2 hist, bin_edges = np.histogram(data_f, bins=100, range=(min, max_rm),
  ↪  density=True)
3 bin_centers = (bin_edges[:-1] + bin_edges[1:]) / 2
4
5 plt.figure(figsize=(8, 5))
```

```
plt.hist(data_f, bins=100, range=(min, max_rm), color='blue',
↪  edgecolor='black', alpha=0.5, density=True, label='Histogram')

gmm_sum = np.zeros_like(bin_edges)
for k in range(3):
    gaussian = pi[k] * norm.pdf(bin_edges, mu[k], np.sqrt(sigma[k]))
    plt.plot(bin_edges, gaussian, label=f'Gaussian {k+1}')
    gmm_sum += gaussian

plt.legend()
plt.title('Histogram with Gaussian Mixture Model Overlay')
plt.show()
```

4 Task 4: Compute weights

Compute the values of $w_{n,k}$ defined by:

$$w_{n,k} = \frac{\mathcal{N}(d_n|\tilde{\mu}_k, \tilde{\sigma}_k^2)\tilde{\pi}_k}{\sum_{k'=1}^{K} \mathcal{N}(d_n|\tilde{\mu}'_k, \tilde{\sigma}_{k'}^2)\tilde{\pi}'_k}.$$

Visualize the values of each $w_{n,k}$ in a figure. You should have three plots which each display only the pixels that belong to the respective class, according to the weights.

Hint:

- The segmentation of a pixel is determined by assigning the class of the highest probability.
- In your visualization, disregard the background pixels by clipping 0

The weights are computed according to equation 6, where the numerator $\pi_k \mathcal{N}(d_n \mid \mu_k, \sigma_k^2)$ represent the likelihood of the observed intensity value d_n under class k, weighted by the prior π_k. The denominator is the sum of these weighted likelihoods over all classes and ensures that the weights form a proper probability distribution.

The values $w_{n,k}$ tell us how likely it is that that pixel n belongs to class k. To visualize the

result, we simply assign each pixel to the class with the highest weight, and show three separate images, one for each class, see Figure 3. Each figure only displays the pixels that are most likely to belong to that class. Background pixels are removed using the brain mask, so that only the relevant regions are shown.

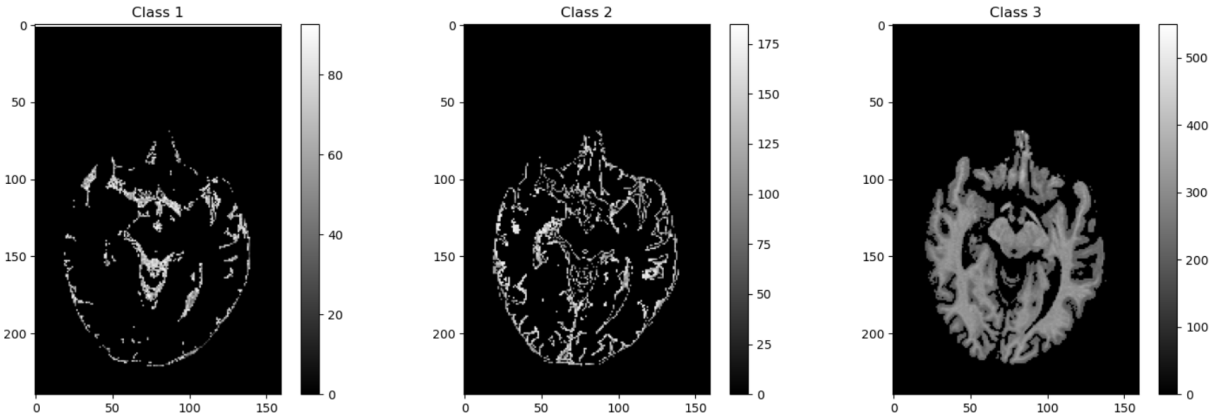

Figure 3: *Hard segmentation obtained by assigning each pixel to the class with the highest weight $w_{n,k}$. Each subplot displays only the pixels belonging to one of the three classes, restricted to the brain mask.*

4.1 Implementation of the code

```
    K = 3
print(data.shape)
H, W = data.shape
likelihoods = np.zeros((H, W, K))
for k in range(K):
    likelihoods[:, :, k] = norm.pdf(data, loc=mu[k], scale=sigma[k])
numerator = likelihoods * pi
denominator = np.sum(numerator, axis=2, keepdims=True)
w = np.divide(numerator,  denominator, out = np.zeros_like(numerator),
 ↪  where=denominator!=0)

plt.figure(figsize=(15, 5))

for k in range(K):
    plt.subplot(1, 3, k+1)
    # Argmax to find the pixel with the maximum prob for that k class
    mask = np.argmax(w, axis=2) == k
    img = np.where(mask, data, 0)
    plt.imshow(img, cmap='gray')
    plt.title(f'Class {k+1}')
```

```
    plt.colorbar()

plt.tight_layout()
plt.show()
```

5 Task 5: Estimate Maximum Likelihood parameters

Estimate the maximum likelihood parameters by iterating between updating the model parameter estimate according to

$$\tilde{\mu_k} \leftarrow \frac{\sum_{n=1}^{N} w_{n,k} d_n}{\sum_{n=1}^{N} w_{n,k}}$$

$$\tilde{\sigma}_k^2 \leftarrow \frac{\sum_{n=1}^{N} w_{n,k} (d_n - \tilde{\mu_k})^2}{\sum_{n=1}^{N} w_{n,k}}$$

$$\tilde{\pi_k} \leftarrow \frac{\sum_{n=1}^{N} w_{n,k}}{N}$$

and by recomputing $w_{n,k}$ according to eq. 3.33 (see Task 4).

Make sure to perform enough iterations (e.g., 100) for the algorithm to converge. As the iterations progress, plot the evolution of the log likelihood function, and update each time the display of $w_{n,k}$ as well as the Gaussian mixture model plot overlaid on the histogram. Include the evolution of the log likelihood function and the plot of the final $w_{n,k}$ and the final mixture model in your report.

Hint:

- The log likelihood can be simply computed by $\log(\sum_k w_k x_k)$

To estimate the maximum likelihood parameters of the Gaussian mixture model, we implemented the expectation-maximization (EM) algorithm in a small `GMM` class (see Appendix). The algorithm alternates between:

- **E-step:** given the current parameters $(\mu_k,\ \sigma_k^2,\ \pi_k)$, we compute the responsibilities $w_{n,k}$

using the numerically stable log-sum-exp formulation.

- **M-step:** we update the means, variances and mixing weights using the weighted formulas in the task description, based only on the non-zero (brain) voxels.

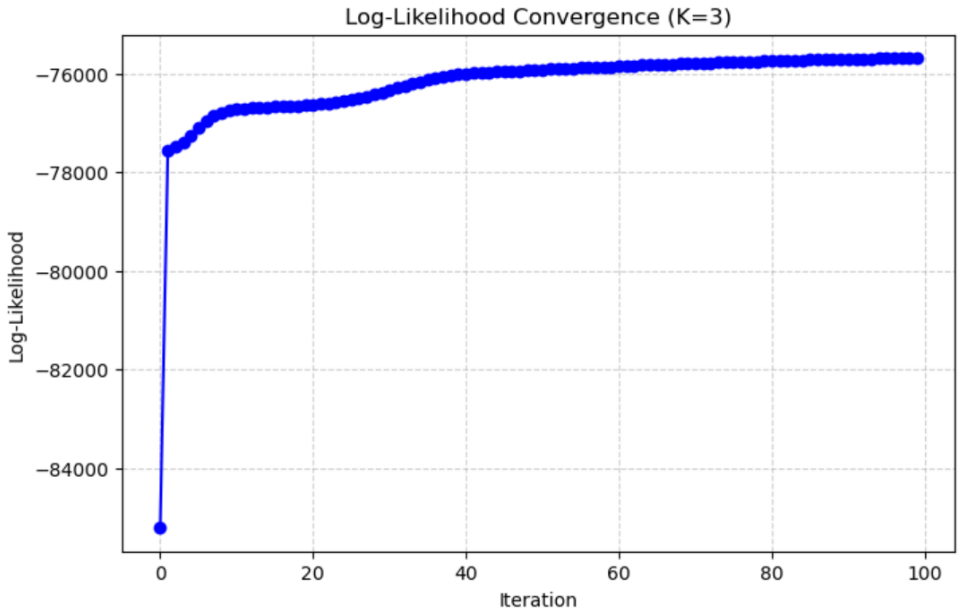

Figure 4: *Evolution of the log-likelihood over EM iterations. The curve increases monotonically and plateaus after a number of iterations, indicating convergence of the parameter estimates.*

We initialize the parameters from the masked data, and run EM for 100 iteration. During the fitting we store the log-likelihood in each of the iterations using only in-mask voxels. Figure 4 shows that the log-likelihood increases monotonically and quickly levels off. This indicates that the algorithm has converged to a stable solution.

After convergence, the three Gaussian components have separated mean intensities, covering low, medium and high intensity ranges that match the expected CSF, gray matter and white matter tissue classes.

Using the final responsibilities $w_{n,k}$, we compute a hard segmentation by assigning each of the voxels to the class with the highest posterior probability. The resulting segmentation maps for the three tissues classes are shown in Figure 5. Compared to the initial segmentation from

Task 4, we can observe how the different tissue types are weighted differently, and more in accordance to the histogram intensity distribution. There are three distinct areas, that matches GM, WM and CSF.

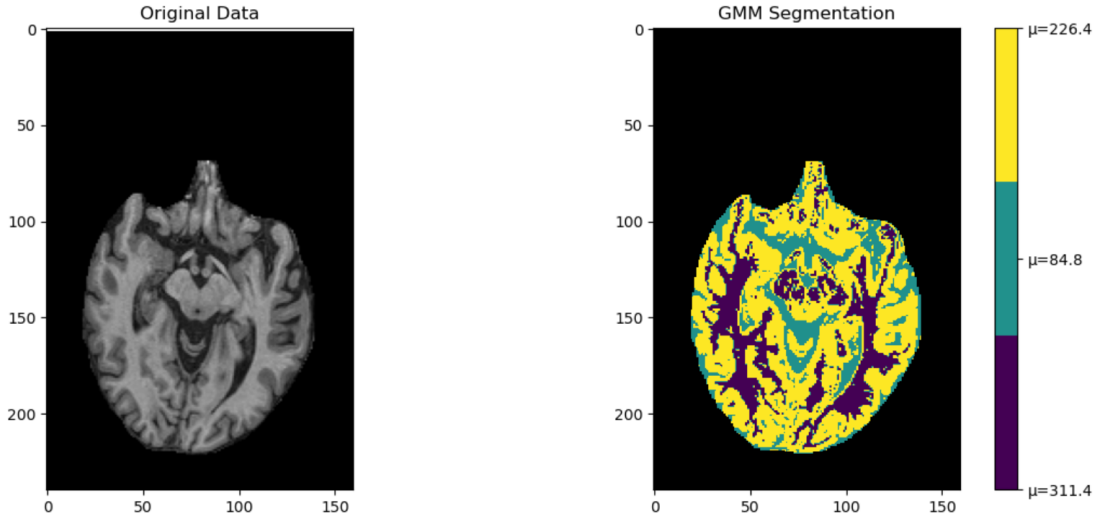

Figure 5: *Final hard segmentation obtained from the converged responsibilities $w_{n,k}$. The left panel shows the original bias-corrected MR slice, and the right panel shows the final GMM-based segmentation. Each colour corresponds to one of the three tissue classes, with the colourbar indicating the class means mu_k in intensity units. Voxels outside the brain mask are shown in black.*

Finally, we overlay the converged Gaussian mixture model on the intensity histogram in Figure 6. Each component is scaled by its mixing weight π_k, and the sum of all three components follows the histogram much more closely than in Task 3. This confirms that the EM algorithm has adapted the model parameters to better fit the observed intensity distribution.

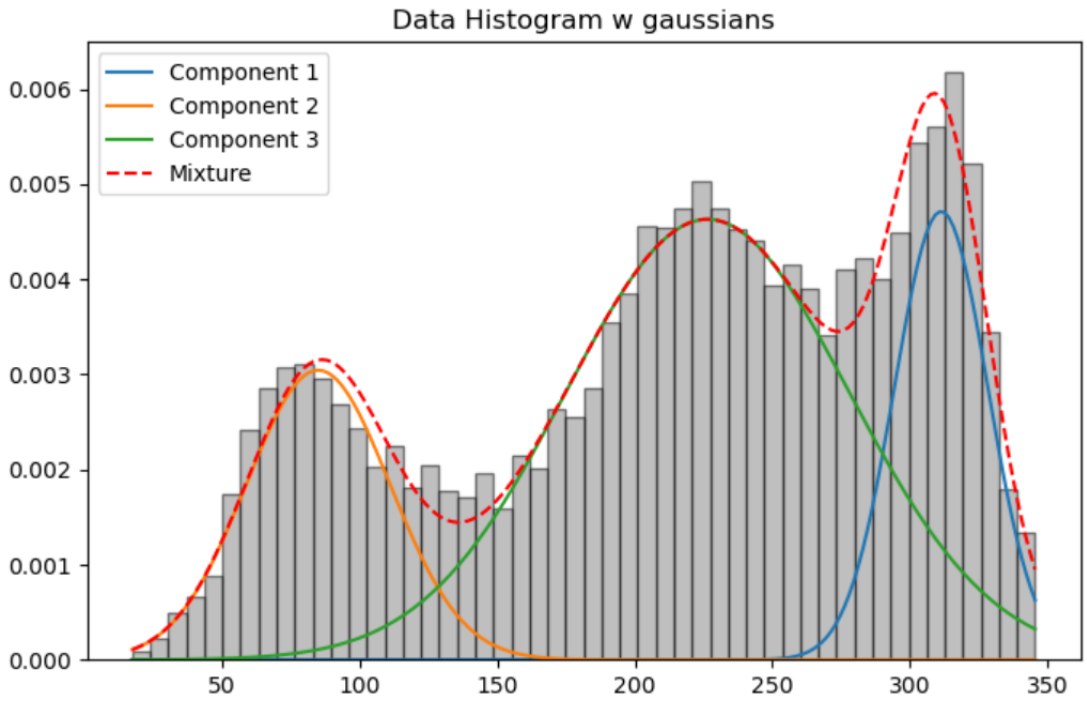

Figure 6: *Corrected intensity histogram with the final Gaussian mixture model overlaid. The three weighted Gaussian components and their sum provide a good approximation of the empirical intensity distribution.*

6 Task 6: Vary only one model parameter

> Keeping all other parameters fixed to their estimated values, vary only μ_2 (the mean of the middle Gaussian distribution) between the estimated values of μ_1 and μ_3 in about 100 steps, and plot for each step the log likelihood function.

To investigate how the log-likelihood behaves when we only change a single parameter, we sweep μ_2 - the mean of the middle Gaussian component - across an evenly spaced range between the estimated values of μ_1 and μ_3 obtained in Task 5. All other parameters $\mu_1,\ \mu_3,\ \sigma_k,\ \pi_k$ are kept fixed.

For each value of μ_2 in this range, we then recompute the log-likelihood of the full GMM model. The resulting curve is shown in Figure 7. The log-likelihood is maximized precisely at the estimated value of μ_2 obtained by the EM algorithm. This shows, that the EM

estimate corresponds to the maximum-likelihood point along this dimension. Additionally, the log-likelihood decreases symmetrically as μ_2 is moved away from its optimum, which reflects that if we were to misplace the middle Gaussian mean results, we would get poorer modeling of the intensity distribution. When checking this, it confirms both the correctness of the EM implementation and the stability of the estimated parameters.

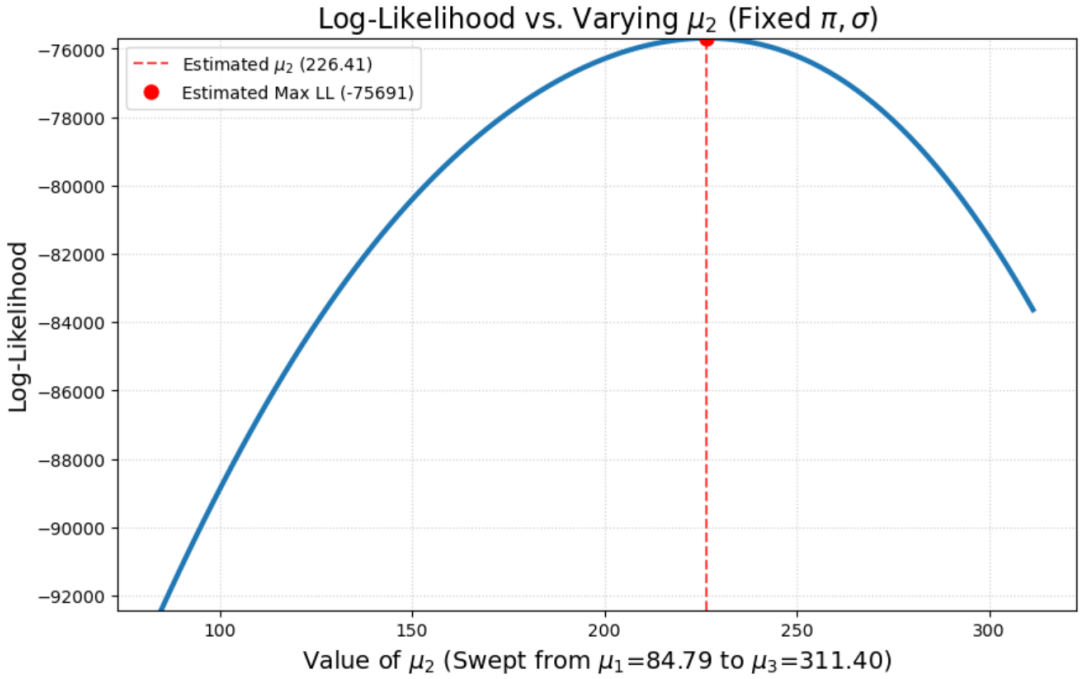

Figure 7: *Log-likelihood as a function of μ_2 while keeping all other parameters fixed.*

7 Task 7: Locate Lower bound

> On the same figure, also plot the lower bound corresponding to the parameter vector in which all parameters are set to their estimated values, except μ_2 which is set to the estimated value of μ_1.

The goal of the task is to calculate the values of the of Q-function, the EM algorithms' lower bound for at specific parameter setting. The setting θ^* is choosen, where all parameter are fixed to the final optimized values from Task 5, except for μ_2, which is set equal to μ_1. By manually calculating the Q-function, for different potential parameter values of μ_2, the lower bound curve

can be visualized (green curve fig. 8).

By first setting $\mu_2 = \mu_1$ it is guaranteed that we are not at the optimal parameter value for μ_2 anymore, and hence, the Q curve should have a clear maximum. Figure 8, shows the Q function for different **potential** values of μ_2, where μ_2 of the **model** was set to $\mu_2 = \mu_1$, from an already converged GMM ($w_{n,k}$ for the converged GMM were recalculated (E-step)). The curve shows how the M-step can choose the optimal new value of μ_2 by maximizing the lower bound given the fixed parameters, and only varying μ_2.

```
    def calculate_elbo_curve(gmm, data, w_frozen, mu_space, mu2_idx):
    """
    Calculates the full ELBO curve for the sweep, using the frozen
    ↪  weights.
    ELBO(ϑ) = Σ_n Σ_k w_nk [ log(π_k) + log(N(x|μ_k,σ_k)) - log(w_nk) ]
    """

    # Get parameters from the converged GMM
    pi_hat = gmm.pi
    mu_hat = gmm.mu
    sigma_hat = gmm.sigma
    eps = gmm.eps

    flat_data_pos = data[data > 0]
    w_frozen_pos = w_frozen[data > 0]

    elbo_values = [] # store lb values
    q_values = [] # store q values

    for mu2_star in mu_space:

        # copy mu and replace mu2 with the swept value
        mu_try = mu_hat.copy()
        mu_try[mu2_idx] = mu2_star

        step_elbo = 0.0
        step_q = 0.0

        for k in range(gmm.k):
            # E_w[log π_k] ->
            log_pi_k = np.log(pi_hat[k] + eps)

            # E_w[log N(x|μ_k, σ_k)]
            log_pdf = norm.logpdf(flat_data_pos,
```

```
                                    loc=mu_try[k],
                                    scale=np.sqrt(sigma_hat[k]))

            # E_w[log w_nk]
            log_w_k = np.log(w_frozen_pos[:, k] + eps)

            # Combine: w * (log(pi) + log(N) - log(w))
            term_k_elbo = w_frozen_pos[:, k] * (log_pi_k + log_pdf -
            ↪  log_w_k) # -> elbo


            step_elbo += np.sum(term_k_elbo)

        elbo_values.append(step_elbo)
    return elbo_values, q_values

def get_frozen_weights(gmm, data, mu_tilde):
    """
    Temporarily sets gmm.mu to mu_tilde, runs the E-step to get the
    frozen responsibilities, and then restores the original gmm.mu.
    """
    # 1. Store the original, converged mu_hat
    mu_original = gmm.mu.copy()

    try:
        gmm.mu = mu_tilde
        w_frozen, _ = gmm.e_step(data)

    finally:
        gmm.mu = mu_original

    return w_frozen
```

8 Task 8: Maximize lower bound

Compute the value for μ_2 that maximizes this lower bound (first line of eq. 3.35), indicate its location on the figure, and comment on the result.

Formulate the lower bound as

$$Q(\theta|\tilde{\theta}) = -\frac{1}{2}\sum_{k=1}^{K}[\frac{1}{\sigma_k^2}\sum_{n=1}^{N} w_{n,k}(d_n - \mu_k - \sum_{m-1}^{M} c_m\phi_{n,m})^2 + (\sum_{n-1}^{N} w_{n,k})\log\sigma_k^2]$$

$$+\sum_{k=1}^{K}[(\sum_{n=1}^{N} w_{n,k})\log\pi_k]$$

$$-\sum_{n=1}^{N}\sum_{k=1}^{K} w_{n,k}\log w_{n,k} - \frac{N}{2}\log(2\pi)$$

The maximum lower bound is calculated and plotted in this task. This task helps in understanding the expectation-maximizating algorithm, specifically, how parameter updates are estimated and why it is robust to local maxima when $\mu_2 = \mu_1$, wrt. convergence properties. In figure 7, the log-likelihood function of varying μ_2 is displayed. There is a clear maximum, but the purpose is to show how the algorithm finds this maximum from the initial conditions and converges to the local maximum. The EM expecation step computes the weight given current parameters $\tilde{\theta}$, the maximization step finds the new parameter $\mathbf{log\ p}(\mathbf{d}|\tilde{\theta})$ that maximize the Q-function, the lower bound plot in task 7, by closed form differentiation. Because the M-step maximizes the lower bound, the weight and parameter updates $\mathbf{log\ p}(\mathbf{d}|\hat{\theta})$ for the next iteration of log-likelihood will be greater or equal to the previous.

The EM algorithm is designed to ensure an "up-hill" climbs toward local maximum and cannot convergence on local minima.

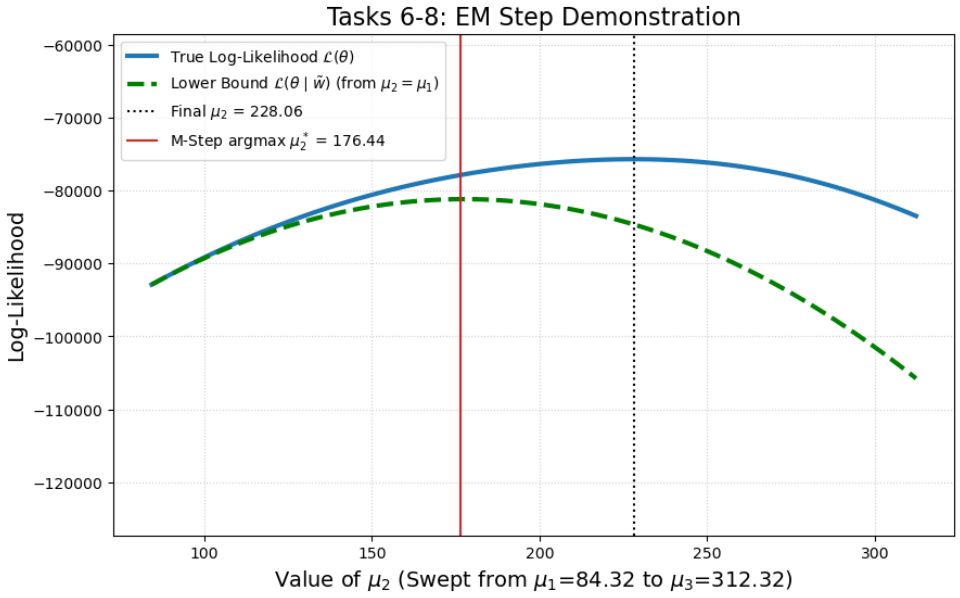

Figure 8: *Lower Bound*

```python
# task 7: what is the maximization of the m_step? It is the maximization of the ELBO/q-finction with frozen weights where mu_2 = mu_1
sorted_indices = np.argsort(gmm.mu)
k_min_index = sorted_indices[0]
mu2_idx = sorted_indices[1] # The index of the middle mean

mu_tilde = gmm.mu.copy()
mu_tilde[mu2_idx] = gmm.mu[k_min_index] # set mu_2 = mu_1


# get frozen weights
w_frozen = get_frozen_weights(gmm, data, mu_tilde)

# task 7: calculate q function- and ELBO curve
elbo_values, q_values = calculate_elbo_curve(gmm, data, w_frozen, mu_space, mu2_idx)


# task 8: find maximum of q function and ELBO
i_star = int(np.argmax(elbo_values))
mu2_star = float(mu_space[i_star])

# Plotting Everything task 6-8
plt.figure(figsize=(10, 6))

# plot 1: true Log-Likelihood
```

```
plt.plot(mu_space, log_likelihoods, color='#1f77b4', linewidth=3,
         label='True Log-Likelihood $\mathcal{L}(\\theta)$')

# plot 2: ELBO
plt.plot(mu_space, elbo_values, color='green', linestyle='--',
 ↪  linewidth=3,
         label='Lower Bound $\mathcal{L}(\\theta \mid \\tilde{w})$ (from
          ↪  $\mu_2=\mu_1$)')

# plot 4: optimal mu2
plt.axvline(mu2_esti, color='k', ls=':', label=f'Final $\mu_2$ =
 ↪  {mu2_esti:.2f}')

# plot 4: argmax of q function and ELBO
plt.axvline(mu2_star, color='tab:red', ls='-',
label=f'M-Step argmax $\mu_2^*$ = {mu2_star:.2f}')

# dynamic y-limits
min_ll_sweep = np.min(log_likelihoods)
max_ll_sweep = np.max(log_likelihoods)
buffer = (max_ll_sweep - min_ll_sweep) * 2
plt.ylim(min_ll_sweep - buffer, max_ll_sweep + buffer/2)

plt.title('Tasks 6-8: EM Step Demonstration', fontsize=16)
plt.xlabel(f'Value of $\\mu_2$ (Swept from $\\mu_1$={mu_min:.2f} to
 ↪  $\\mu_3$={mu_max:.2f})', fontsize=14)
plt.ylabel('Log-Likelihood', fontsize=14)
plt.grid(True, linestyle=':', alpha=0.6)
plt.legend()
plt.show()

print(f"Lower bound maximized at μ2* = {mu2_star:.4f}")

```

Conclusion

During this exercise, we have learned how we can use a Gaussian Mixture Model for segmentation of medical images.

We have focused on how, going from an initial guess to applying the EM algorithm, demonstrating how the algorithm iteratively estimates the parameters needed for accurate

segmentation.

Finally, we also looked at the lower bound and maximization step in detail. We learned and visualized how the model learns the "best" parameters with the EM algorithm.

Overall, the tasks gave a practical and intuitive understanding of GMM-based segmentation and the mechanics of the EM algorithm.

9 Appendix A

```
import numpy.ma as ma
class GMM:
    def __init__(self, n_iterations=100, k=3):
        """
        Initiate class
        Defines imported variables

        Parameter
        ---------
        self:
            object to store variables and functions

        n_iterations:
            Number of iteration before termination

        k:
            Number of class to estimate with gaussians

        """
        self.n_iter = n_iterations
        self.k = k

        self.mu = None
        self.sigma = None
        self.pi = None

        self.log_likelihood_history = []
        self.last_w = None
        self.eps = 1e-12

    def init_param(self, data):
        """
        Initiates parameters
            Flatten data, pi, mu, sigma, min, max and max_rm of the data

        Parameters
        ----------
        Self

        date:
            Any HxW data
        """

```

```
        self.flat_data = data[data>0]
        N = len(self.flat_data)

        self.pi = np.ones(self.k)/self.k

        sample = np.random.choice(N, self.k, replace=False)
        self.mu = self.flat_data[sample]

        init_var = np.var(self.flat_data)/self.k
        self.sigma = np.ones(self.k) * init_var

        self.min = np.min(self.flat_data)
        self.max = np.max(self.flat_data)
        self.max_rm = np.percentile(self.flat_data, 99)

    def e_step(self, data):
        """
        Estiamation step. This step computes the likehood to calculate
        ↪ the weights w and finally computes the totel likelihood and
        ↪ log likelihood

        Parameters
        ----------

        self

        data

        Returns
        -----------
        w:
            Computed weights

        Log likelihood:
            The total log likelihood of this step
        """
        H, W = data.shape
        log_comp = np.zeros((self.k, H, W))

        for k in range(self.k):
            log_pdf = norm.logpdf(data, loc=self.mu[k],
            ↪ scale=np.sqrt(self.sigma[k]))
            log_comp[k, :, :] = np.log(self.pi[k] + self.eps) + log_pdf
        m = np.max(log_comp, axis=0, keepdims=True) # shape (1, H, W)
```

```
        log_sum_exp = m + np.log(np.sum(np.exp(log_comp - m), axis=0,
        ↪  keepdims=True) + self.eps)

        # w = exp( log(numerator) - log(denominator) )
        w = np.exp(log_comp - log_sum_exp) # shape (K, H, W)
        log_likelihood = np.sum(log_sum_exp[0, data > 0])
        w = w.transpose(1, 2, 0) # from (K, H, W) to (H, W, K)

        self.last_w = w
        return w, float(log_likelihood)

    def m_step(self, data, w):
        """
        Maximization step: updates the parameters mu, sigma, and pi.
        """
        flat_data_pos = data[data > 0]
        w_pos = w[data > 0]
        N_pos = len(flat_data_pos)

        for k in range(self.k):
            w_k = w_pos[:, k]
            w_k_sum = np.sum(w_k)

            if w_k_sum > 0:
                self.mu[k] = np.sum(w_k * flat_data_pos) / w_k_sum

                new_sigma = np.sum(w_k * (flat_data_pos - self.mu[k]) **
                ↪  2) / w_k_sum
                self.sigma[k] = np.maximum(new_sigma, self.eps)

                self.pi[k] = w_k_sum / N_pos
            else:
                self.pi[k] = 0.0

        sum_pi = np.sum(self.pi)
        if sum_pi > 0:
            self.pi /= sum_pi
        else:
            self.pi = np.ones(self.k) / self.k

    def fit(self, data):

        self.init_param(data)

        for i in range(self.n_iter):
```

```
            w, log_likelihood = self.e_step(data)
            self.log_likelihood_history.append(log_likelihood)
            self.m_step(data, w)

        return self
# ---------------------------------------------- plotting functions ----------------------------------------------------

    def plot_convergence(self):
        """Plots the log-likelihood history to show convergence."""
        if not self.log_likelihood_history:
            print("Model has not been fitted yet. Run .fit(data) first.")
            return

        plt.figure(figsize=(8, 5))
        plt.plot(self.log_likelihood_history, marker='o', linestyle='-', color='blue')
        plt.title(f'Log-Likelihood Convergence (K={self.k})')
        plt.xlabel('Iteration')
        plt.ylabel('Log-Likelihood')
        plt.grid(True, linestyle='--', alpha=0.6)
        plt.show()

    def plot_segmentation_components(gmm_model, data):
        """
        Plots the hard segmentation masks for each component (K=3).
        """
        # get segmentation
        # shape (H, W) where each value is 0, 1, or 2
        segmentation_map = np.argmax(gmm_model.last_w, axis=2)

        background_mask = (data <= 0)

        fig, axes = plt.subplots(1, gmm_model.k, figsize=(15, 5))

        for k in range(gmm_model.k):
            ax = axes[k]

            # one if pixel belongs to component k
            binary_mask = (segmentation_map == k)

            # only show the segmented tissue (data > 0)
            plot_data = ma.array(binary_mask.astype(int), mask=background_mask)

```

```
            cmap = plt.cm.gray.copy()
            cmap.set_bad('black')

            im = ax.imshow(plot_data, cmap=cmap, vmin=0, vmax=1)

            mu_val = gmm_model.mu[k]
            ax.set_title(f'Comp {k} Hard Mask: $\\mu$={mu_val:.2f}')
            ax.axis('off')
            ax.axis('off')

        plt.tight_layout()
        plt.show()

    def plot_responsibilities(self, data):
        if self.last_w is None:
            print("Model has not been fitted yet. Run .fit(data) first.")
            return

        total = self.k + 1
        ncols = 3 if total > 3 else (2 if total > 1 else 1)
        nrows = int(np.ceil(total / ncols))

        fig, axes = plt.subplots(nrows, ncols, figsize=(4 * ncols, 4 *
        ↪  nrows))
        axes = np.array(axes).flatten()

        axes[0].imshow(data, cmap='gray')
        axes[0].set_title('Original')
        axes[0].axis('off')

        mask = (data <= 0)
        cmap = plt.cm.get_cmap('gray_r').copy()
        cmap.set_bad('black')

        for idx in range(self.k):
            ax = axes[idx + 1]
            im = ax.imshow(ma.array(self.last_w[:, :, idx], mask=mask),
            ↪  cmap=cmap, vmin=0, vmax=1)
            ax.set_title(f'Comp {idx}: μ={self.mu[idx]:.2f},
            ↪  σ={self.sigma[idx]:.2f}')
            ax.axis('off')
            fig.colorbar(im, ax=ax, fraction=0.046, pad=0.04)

        for ax in axes[total:]:
            fig.delaxes(ax)
```

```
        plt.tight_layout()
        plt.show()

    def plot_segmentation(self, data):
        """
        Plots the original image and GMM segmentation side-by-side.
        """
        if self.last_w is None:
            print("Model has not been fitted yet. Run .fit(data) first.")
            return

        # Create side-by-side plots
        fig, (ax1, ax2) = plt.subplots(1, 2, figsize=(12, 5))

        # Plot original data
        ax1.imshow(data, cmap='gray')
        ax1.set_title('Original Data')

        # Create segmentation map
        seg_map = np.argmax(self.last_w, axis=2)
        mask = (data <= 0)
        masked_seg = ma.array(seg_map, mask=mask)

        # Plot segmentation with meaningful colors
        cmap = plt.cm.get_cmap('viridis', self.k)
        cmap.set_bad(color='black')
        im = ax2.imshow(masked_seg, cmap=cmap)
        ax2.set_title('GMM Segmentation')

        # Add colorbar with component info
        cbar = plt.colorbar(im, ax=ax2)
        cbar.set_ticks(range(self.k))
        cbar.set_ticklabels([f'μ={self.mu[i]:.1f}' for i in
        ↪  range(self.k)])

        plt.tight_layout()
        plt.show()

    def plot_hist(self):
        """Plots histogram of data with GMM component overlay"""
        plt.figure(figsize=(8, 5))

        # Plot histogram of data
        plt.hist(self.flat_data, bins=50, range=(self.min, self.max_rm),
```

```
                density=True, alpha=0.5, color='gray', edgecolor='black')

        # Plot GMM components
        x = np.linspace(self.min, self.max_rm, 200)
        total = np.zeros_like(x)
        for k in range(self.k):
            component = self.pi[k] * norm.pdf(x, self.mu[k],
            ↪  np.sqrt(self.sigma[k]))
            plt.plot(x, component, label=f'Component {k+1}')
            total += component

        plt.plot(x, total, 'r--', label='Mixture')
        plt.title('Data Histogram w gaussians')
        plt.legend()
        plt.show()
```